

PRODUCT REVIEW AND RATING MANAGEMENT SYSTEM

Project Title: Flipkart E-Commerce project

1. Database Design

1.1 Review Table

Entity:Review

Attribute	Key type
Review_ID	Primary key
Customer_ID	Foreign key
Product_ID	Foreign key
Rating	-
Review_Text	-

1.2 Rating Table

Attribute	Key type
Rating_ID	Primary key
Customer_ID	Foreign key
Product_ID	Foreign key
Rating	-

2. Table Relationships

- **Customer → Review:** One customer can write many reviews.
- **Product → Review:** One product can have many reviews.
- **Review → Rating:** One review can have one rating.
- **Product → Rating:** One product can receive many ratings.

3. Table Creation

3.1 Create Review Table

```
SQL> CREATE TABLE Review (  
2   Review_ID NUMBER PRIMARY KEY,  
3   Customer_ID NUMBER,  
4   Product_ID NUMBER,  
5   Review_Text VARCHAR2(255),  
6   Review_Date DATE,  
7   FOREIGN KEY (Customer_ID) REFERENCES Customer(Customer_ID),  
8   FOREIGN KEY (Product_ID) REFERENCES Product(Product_ID)  
9 );  
Table created.
```

3.2 Create Rating Table

```
SQL> CREATE TABLE Rating (  
  2     Rating_ID NUMBER PRIMARY KEY,  
  3     Customer_ID NUMBER,  
  4     Product_ID NUMBER,  
  5     Rating NUMBER(1),  
  6     FOREIGN KEY (Customer_ID) REFERENCES Customer(Customer_ID),  
  7     FOREIGN KEY (Product_ID) REFERENCES Product(Product_ID)  
  8 );
```

Table created.

4. Review Data Insertion

```
SQL> INSERT INTO Review VALUES  
  2 (701, 1, 101, 'Excellent laptop with good performance', SYSDATE);  
  
1 row created.  
  
SQL>  
SQL> INSERT INTO Review VALUES  
  2 (702, 2, 102, 'Good smartphone with useful features', SYSDATE);  
  
1 row created.  
  
SQL>  
SQL> INSERT INTO Review VALUES  
  2 (703, 3, 103, 'Good quality T-Shirt', SYSDATE);  
  
1 row created.  
  
SQL>  
SQL> INSERT INTO Review VALUES  
  2 (704, 4, 104, 'The mixer works well', SYSDATE);  
  
1 row created.  
  
SQL>  
SQL> INSERT INTO Review VALUES  
  2 (705, 5, 105, 'Very useful book for Python learning', SYSDATE);  
  
1 row created.
```

5. Rating Data Insertion

```

SQL> INSERT INTO Rating VALUES
  2 (801, 1, 101, 5);

1 row created.

SQL>
SQL> INSERT INTO Rating VALUES
  2 (802, 2, 102, 4);

1 row created.

SQL>
SQL> INSERT INTO Rating VALUES
  2 (803, 3, 103, 4);

1 row created.

SQL>
SQL> INSERT INTO Rating VALUES
  2 (804, 4, 104, 3);

1 row created.

SQL>
SQL> INSERT INTO Rating VALUES
  2 (805, 5, 105, 5);

1 row created.

```

6. Retrieve Product Review Details

```

SQL> SELECT
  2 r.Review_ID,
  3 c.Customer_ID,
  4 c.First_Name || ' ' || c.Last_Name AS Customer_Name,
  5 p.Product_ID,
  6 p.Product_Name,
  7 r.Review_Text,
  8 r.Review_Date
  9 FROM Review r
 10 JOIN Customer c
 11   ON r.Customer_ID = c.Customer_ID
 12 JOIN Product p
 13   ON r.Product_ID = p.Product_ID
 14 ORDER BY r.Review_ID;

REVIEW_ID CUSTOMER_ID
-----
CUSTOMER_NAME
-----
PRODUCT_ID
-----
PRODUCT_NAME
-----
REVIEW_TEXT
-----
REVIEW_DA
-----
      701              1
REVIEW_ID CUSTOMER_ID
-----
CUSTOMER_NAME
-----
PRODUCT_ID
-----
PRODUCT_NAME
-----
REVIEW_TEXT
-----
REVIEW_DA
-----
Renu Srinivasan
REVIEW_ID CUSTOMER_ID
-----
CUSTOMER_NAME
-----
PRODUCT_ID
-----
PRODUCT_NAME
-----
REVIEW_TEXT
-----
REVIEW_DA
-----
      101
REVIEW_ID CUSTOMER_ID
-----
CUSTOMER_NAME
-----
PRODUCT_ID
-----
PRODUCT_NAME
-----
REVIEW_TEXT
-----
REVIEW_DA
-----
Laptop
REVIEW_ID CUSTOMER_ID
-----
CUSTOMER_NAME
-----
PRODUCT_ID
-----
PRODUCT_NAME
-----
REVIEW_TEXT
-----

```

7. Calculate Average Product Ratings

This uses the AVG() aggregate function.

```

SQL> SELECT
2     p.Product_ID,
3     p.Product_Name,
4     ROUND(AVG(rt.Rating), 2) AS Average_Rating
5 FROM Product p
6 JOIN Rating rt
7     ON p.Product_ID = rt.Product_ID
8 GROUP BY p.Product_ID, p.Product_Name
9 ORDER BY Average_Rating DESC;

```

```

PRODUCT_ID
-----
PRODUCT_NAME
-----
AVERAGE_RATING
-----
      105
Python Programming Book
      5

      101
Laptop
      5

PRODUCT_ID
-----
PRODUCT_NAME
-----
AVERAGE_RATING
-----
      102
Smartphone
      4

      103
T-Shirt

PRODUCT_ID
-----
PRODUCT_NAME
-----
AVERAGE_RATING
-----
      4

      104
Mixer Grinder
      3

```

```

SQL> SELECT p.Product_Name,
2     ROUND(AVG(r.Rating), 2) AS Average_Rating
3 FROM Product p
4 JOIN Review r
5     ON p.Product_ID = r.Product_ID
6 GROUP BY p.Product_Name;

```

```

PRODUCT_NAME
-----
AVERAGE_RATING
-----
Laptop

Smartphone

T-Shirt

PRODUCT_NAME
-----
AVERAGE_RATING
-----
Mixer Grinder

Python Programming Book

```

8. Identify Highly Rated Products

```
SQL> SELECT
2     p.Product_ID,
3     p.Product_Name,
4     ROUND(AVG(rt.Rating), 2) AS Average_Rating
5 FROM Product p
6 JOIN Rating rt
7     ON p.Product_ID = rt.Product_ID
8 GROUP BY p.Product_ID, p.Product_Name
9 HAVING AVG(rt.Rating) >= 4.5
10 ORDER BY Average_Rating DESC;
```

PRODUCT_ID

PRODUCT_NAME

AVERAGE_RATING

105
Python Programming Book
5

101
Laptop
5

PRODUCT_ID

PRODUCT_NAME

AVERAGE_RATING

9. Product Rating Report

```
SQL> SELECT
2     p.Product_ID,
3     p.Product_Name,
4     COUNT(rt.Rating_ID) AS Total_Ratings,
5     ROUND(AVG(rt.Rating), 2) AS Average_Rating,
6     MIN(rt.Rating) AS Lowest_Rating,
7     MAX(rt.Rating) AS Highest_Rating
8 FROM Product p
9 JOIN Rating rt
10    ON p.Product_ID = rt.Product_ID
11 GROUP BY p.Product_ID, p.Product_Name
12 ORDER BY Average_Rating DESC;
```

PRODUCT_ID

PRODUCT_NAME

TOTAL_RATINGS AVERAGE_RATING LOWEST_RATING HIGHEST_RATING

105
Python Programming Book
1 5 5 5

101
Laptop
1 5 5 5

PRODUCT_ID

PRODUCT_NAME

TOTAL_RATINGS AVERAGE_RATING LOWEST_RATING HIGHEST_RATING

102
Smartphone
1 4 4 4

103
T-Shirt

PRODUCT_ID

PRODUCT_NAME

TOTAL_RATINGS AVERAGE_RATING LOWEST_RATING HIGHEST_RATING

1 4 4 4

104
Mixer Grinder
1 3 3 3